

Experiment no: 02

Experiment name: To develop sine and cosine function wave-forms.

Theory:

A discrete-time sinusoidal signal is generated by taking samples of a continuous waveform at regular sample intervals T_s . The general mathematical representations for discrete sine and cosine functions are:

1. Discrete Sine Waveform: $x[n] = \sin \omega n$
2. Discrete Cosine Waveform: $x[n] = \cos \omega n = \sin(\omega n + \frac{\pi}{2})$

Where:

- n = Discrete sample index ($n = 0, 1, 2, 3, \dots$)
- ω = Normalized digital angular frequency in radians per sample.

The two waveforms share identical amplitude and frequency characteristics, but the cosine function leads the sine function by a fixed phase angle of 90° ($\frac{\pi}{2}$ radians). This phase shift is governed by the trigonometric identity $\cos \theta = \sin(\theta + \frac{\pi}{2})$, which causes the sine sequence to start at 0 while the cosine sequence starts at its maximum value of 1 when $n = 0$.

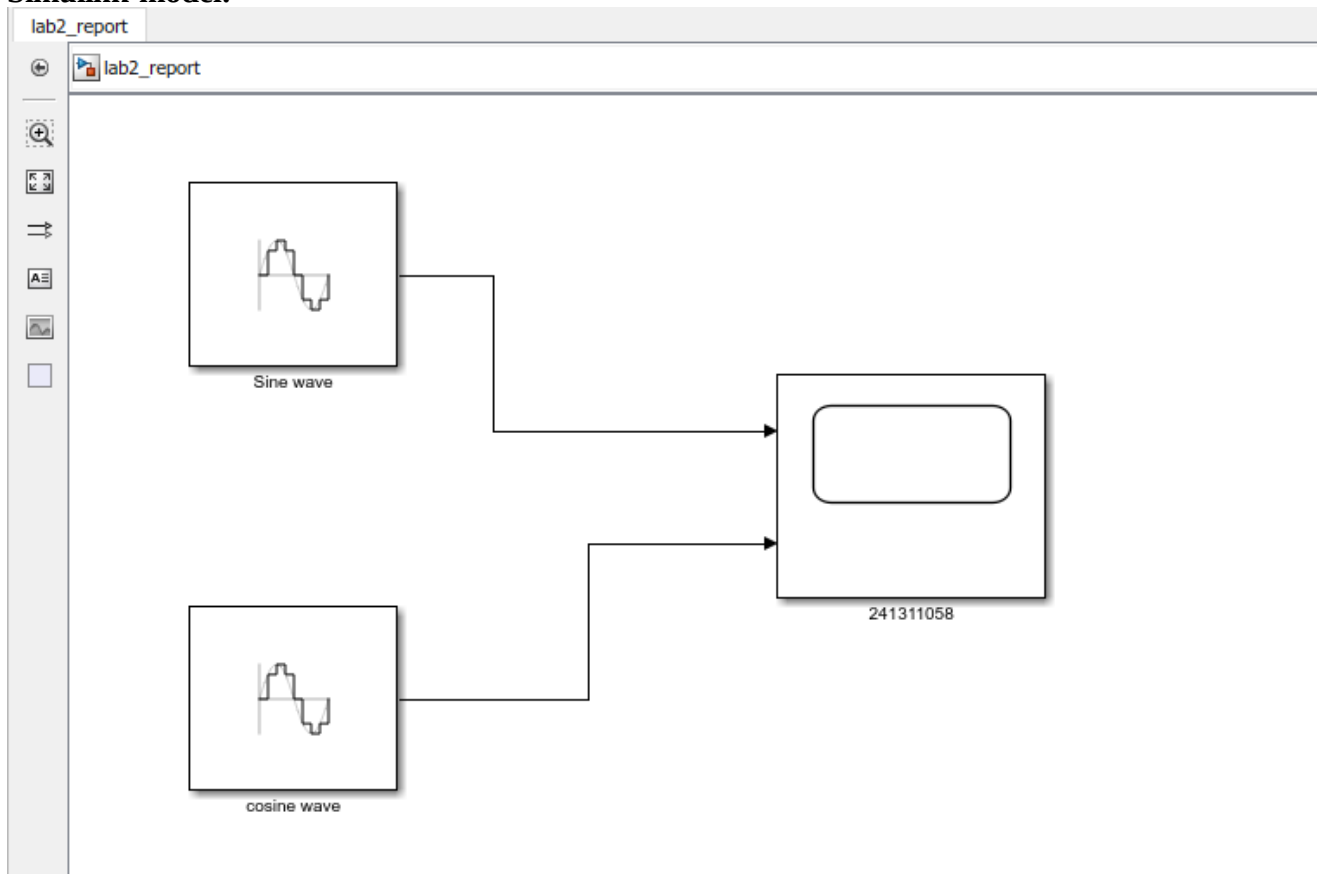
DSP tools:

MATLAB 2019: Simulink Model Setup

Required Blocks

- Sine Wave (from *Simulink / Sources*) $\rightarrow$ Rename to Sine_Wave
- Sine Wave (from *Simulink / Sources*) $\rightarrow$ Rename to Cosine_Wave
- Scope (from *Simulink / Sinks*)

Simulink model:



Parameters:

Block Parameters: Sine wave

Sine Wave

Output a sine wave:

$$O(t) = \text{Amp} * \sin(\text{Freq} * t + \text{Phase}) + \text{Bias}$$

Sine type determines the computational technique used. The parameters in the two types are related through:

Samples per period = $2 * \pi / (\text{Frequency} * \text{Sample time})$

Number of offset samples = $\text{Phase} * \text{Samples per period} / (2 * \pi)$

Use the sample-based sine type if numerical problems due to running for large times (e.g. overflow in absolute time) occur.

Parameters

Sine type: Time based

Time (t): Use simulation time

Amplitude: 1

Bias: 0

Frequency (rad/sec): $2 * \pi * 7$

Phase (rad): 0

Sample time: $1e-4$

?

OK

Cancel

Help

Apply

Block Parameters: cosine wave

Sine Wave

Output a sine wave:

$$O(t) = \text{Amp} * \sin(\text{Freq} * t + \text{Phase}) + \text{Bias}$$

Sine type determines the computational technique used. The parameters in the two types are related through:

Samples per period = $2 * \pi / (\text{Frequency} * \text{Sample time})$

Number of offset samples = $\text{Phase} * \text{Samples per period} / (2 * \pi)$

Use the sample-based sine type if numerical problems due to running for large times (e.g. overflow in absolute time) occur.

Parameters

Sine type: Time based

Time (t): Use simulation time

Amplitude: 1

Bias: 0

Frequency (rad/sec): $2 * \pi * 7$

Phase (rad): $\pi / 2$

Sample time: $1e-4$

?

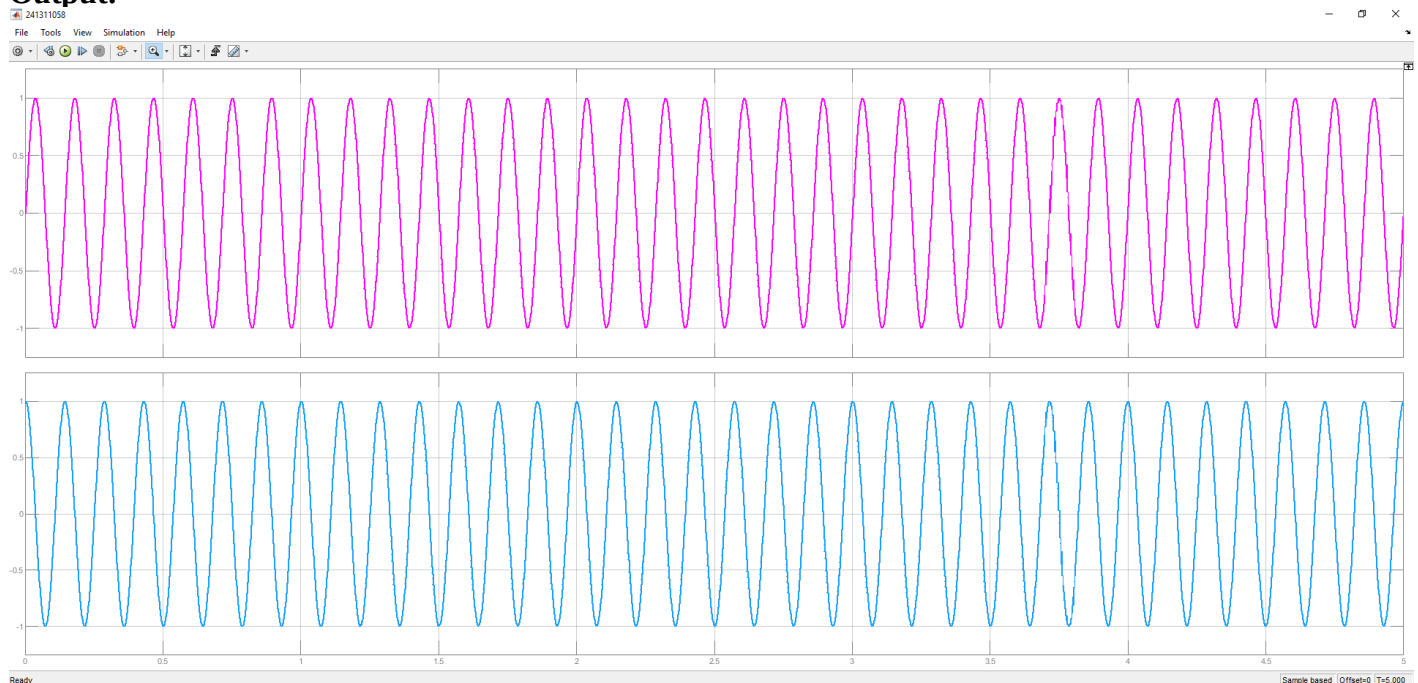
OK

Cancel

Help

Apply

Output:



Result and Discussion:

The simulation successfully generated discrete sine and cosine waveforms using two Sine Wave blocks and a two-port Scope in MATLAB R2019b. The signal connected to Port 1 ($x[n] = \sin \omega n$) displayed a periodic sinusoidal wave starting at an amplitude of 0 at time $t = 0$, whereas the signal connected to Port 2 ($x[n] = \cos \omega n = \sin(\omega n + \frac{\pi}{2})$) began at its positive peak amplitude of 1 at $t = 0$. Observing both traces simultaneously on the Scope confirmed that the cosine signal leads the sine signal by a constant phase angle of 90° ($\frac{\pi}{2}$ radians), directly validating the mathematical identity $\cos \theta = \sin(\theta + \frac{\pi}{2})$. Furthermore, setting a discrete sample time of 0.01 seconds ensured smooth sampling across the 5-second simulation without introducing aliasing or distortion.

Precaution:

When configuring the Simulink model, ensure that the discrete sample time in both Sine Wave blocks is set to an identically small value (such as 0.01 seconds) to satisfy the Nyquist-Shannon sampling criterion and avoid visual aliasing or distorted waveforms. Additionally, strictly verify that the phase parameter for the cosine wave block is entered in radians as $\pi/2$ rather than degrees (90), as Simulink interprets phase inputs in radians by default. Finally, ensure the Scope is updated to accept 2 input ports prior to running the simulation, and clear any previous simulation cache in MATLAB to prevent output plotting errors.